# ECE 590
# Neural-Network-Based Large Language Models

Tuesdays and Thursdays 1:25-2:40
Fitzpatrick Schiciano A 1464

# ECE 590
# Neural-Network-Based Large Language Models

Lecture 5
01/22/2026

# Mathematical Descriptions

**Gradient Descent**

1. Start with an initial $\boldsymbol{\theta}_0$
2. Calculate gradient $\nabla f(\boldsymbol{\theta}_k)$ over *all* data
3. Iteratively update: $\boldsymbol{\theta}_{K+1} \leftarrow \boldsymbol{\theta}_k - \alpha_k \nabla f(\boldsymbol{\theta}_k)$
4. Repeat 2-3 until solution is good enough

**Stochastic Gradient Descent**

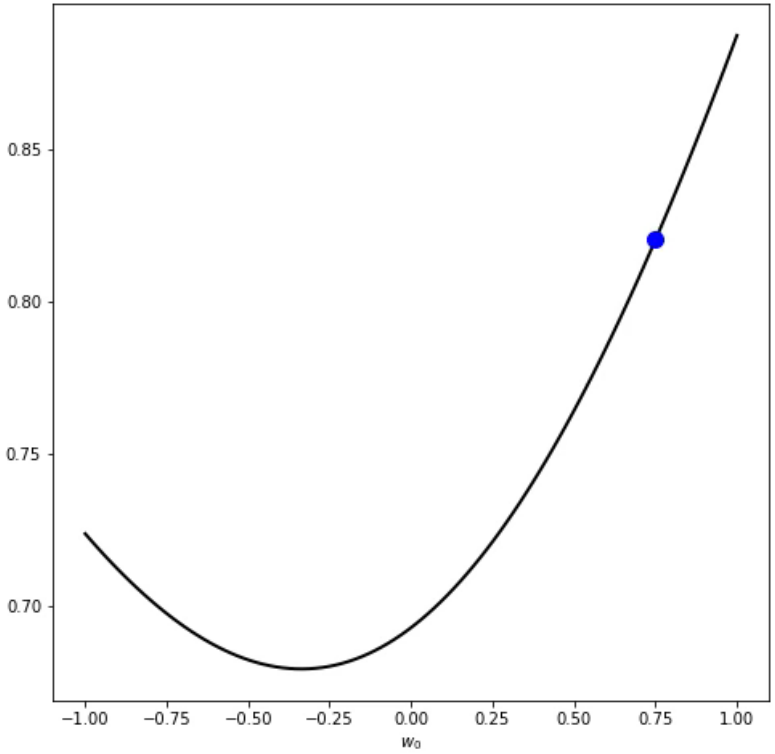
1. Start with an initial $\boldsymbol{\theta}_0$
2. Choose a random mini-batch of data
3. Estimate gradient $\widehat{\nabla f}(\boldsymbol{\theta}_k)$ using mini-batch
4. Iteratively update: $\boldsymbol{\theta}_{k+1} \leftarrow \boldsymbol{\theta}_k - \alpha_k \widehat{\nabla f}(\boldsymbol{\theta}_k)$
5. Repeat 2-4 until solution is good enough

# Why mini batches?

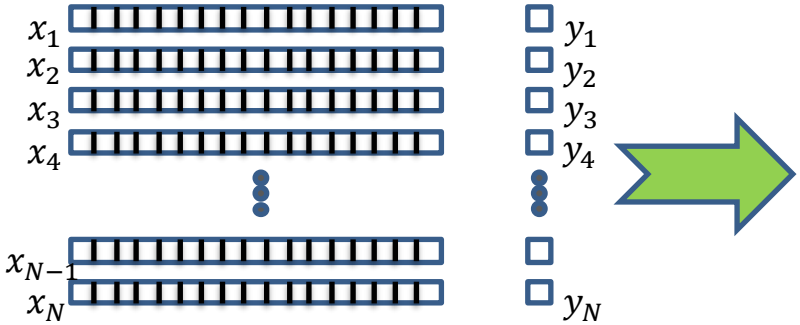
- We can reduce the noise of the gradient estimator
- We can calculate gradients in parallel (with GPUs)
- Interestingly: mini batch size usually depends on the memory capacity of the GPU

# Revisiting our Visualization

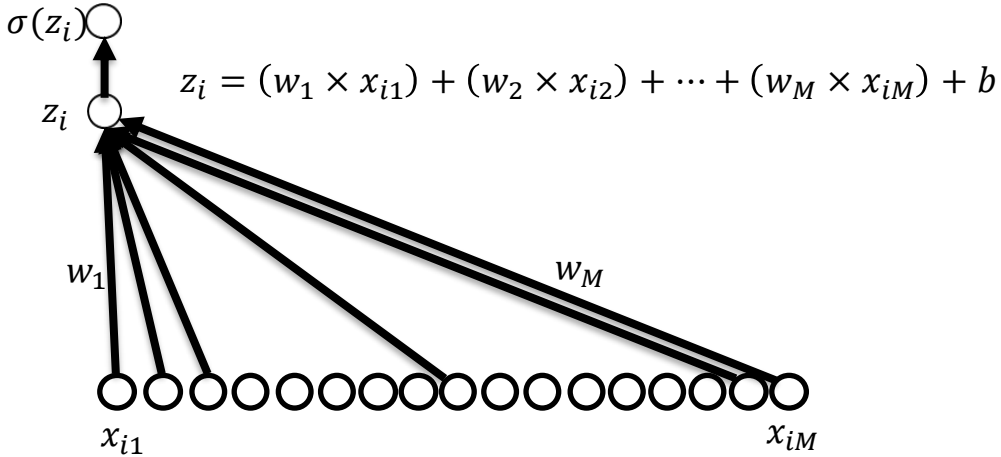
- When the starting point is “far away” from the optimal, then we typically move in the right direction
- When the current point is near the optimal value, it is very random whether we move in the correct direction or not



# Learning Model Parameters (Recap)




Training Set




Logistic Regression Model (or "Network")

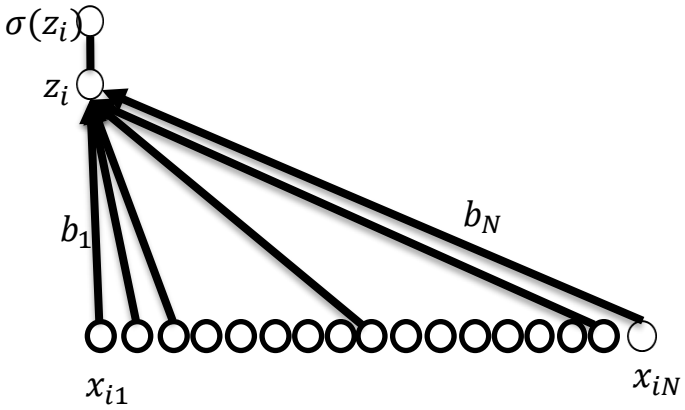
Stochastic Gradient Descent

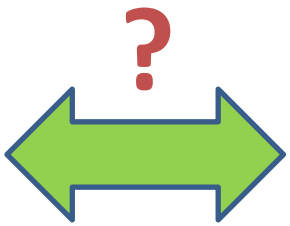
Learned Parameters $\theta = (b, w_1, \ldots w_M)$

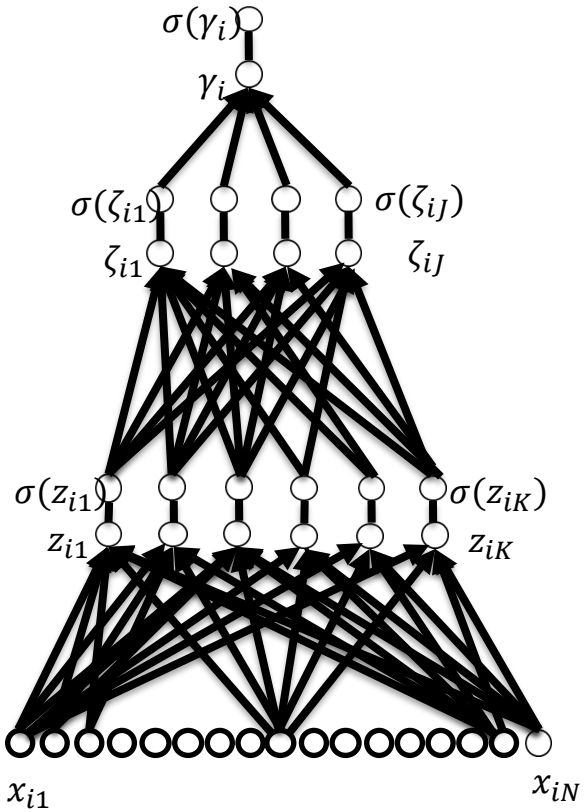
An Introduction to Model Validation

# ESTIMATING PERFORMANCE

# Do we always want to increase complexity?



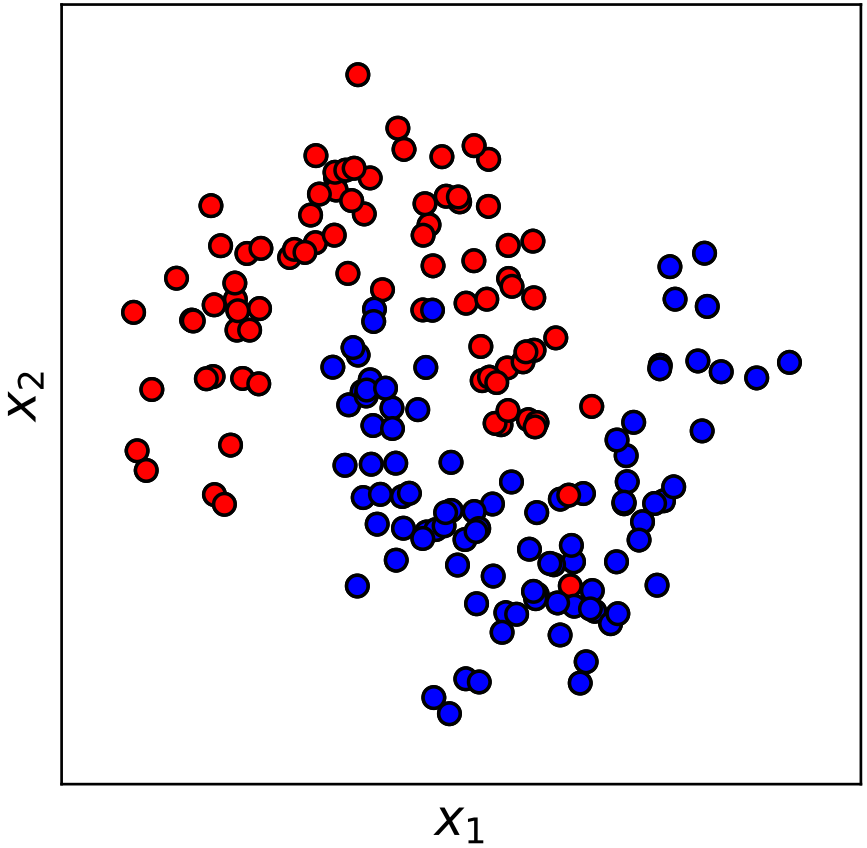



# Creating deep models can help us learn complex relationships

My learning multiple layers and transformation, it is possible to have a non-linear classifier capable of more accurately capturing the data.

However, a deep model can also give “perfect” performance on the training dataset and **fail completely** in the real world

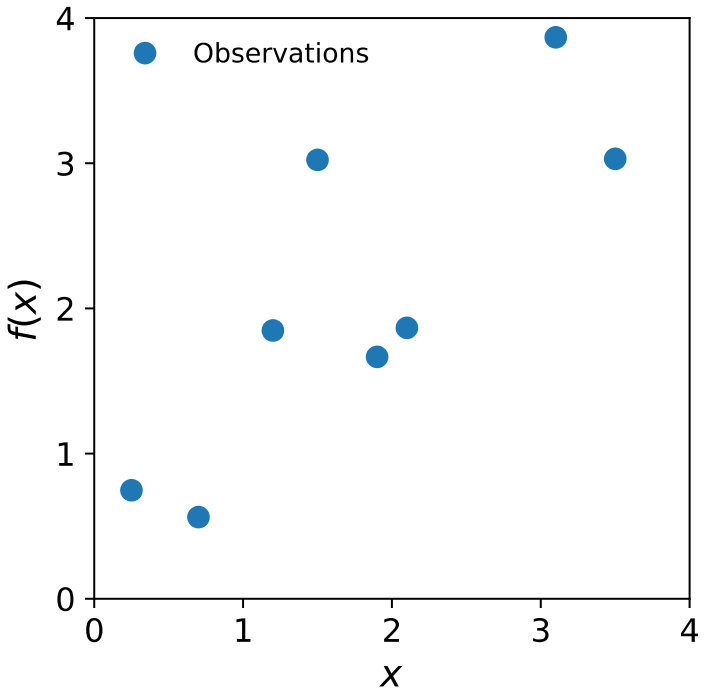
We need to *validate* the performance



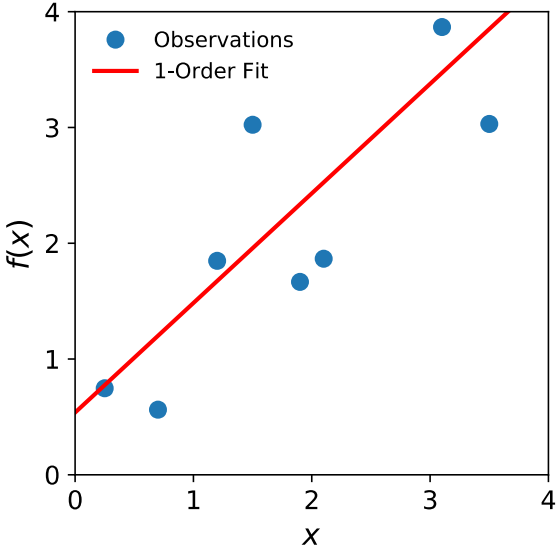
# Overfitting

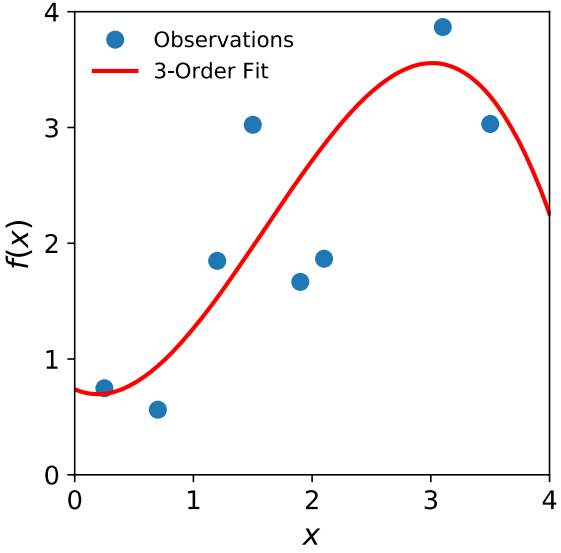
"Overfitting" is when the learned model increases complexity to fit the observed training data *too well* – will not work to predict future data!

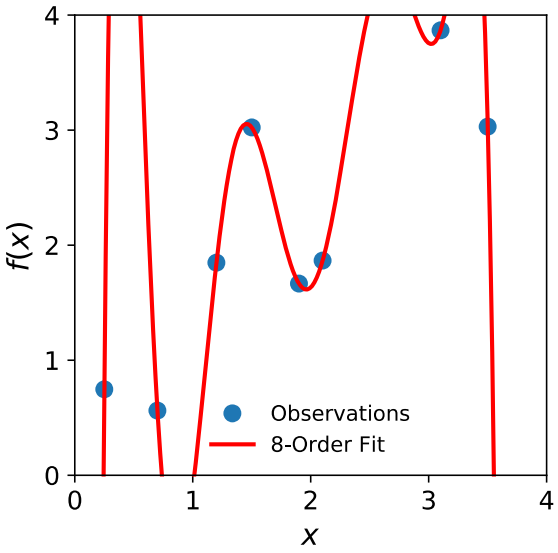
What would we want to use to fit these example data points?



# Classic Example: Increasing Polynomial Order



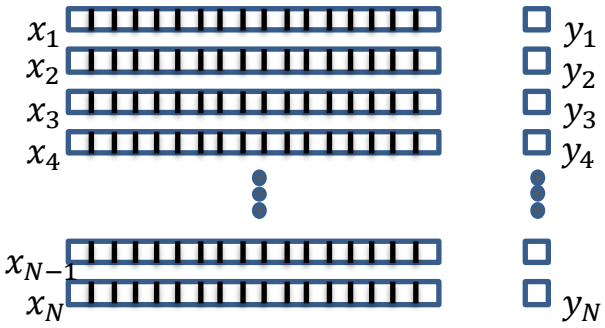



# What happens in overfitting?

- We are increasing the number of parameters in the model, which means:
  - More parameters to estimate (all of their errors add up)
  - Can learn *complex* relationships, maybe too complex for reality

- When we *overfit*, this means that we will not *generalize*
  - We want our models and analysis to generalize, or provide accurate predictions on newly collected data



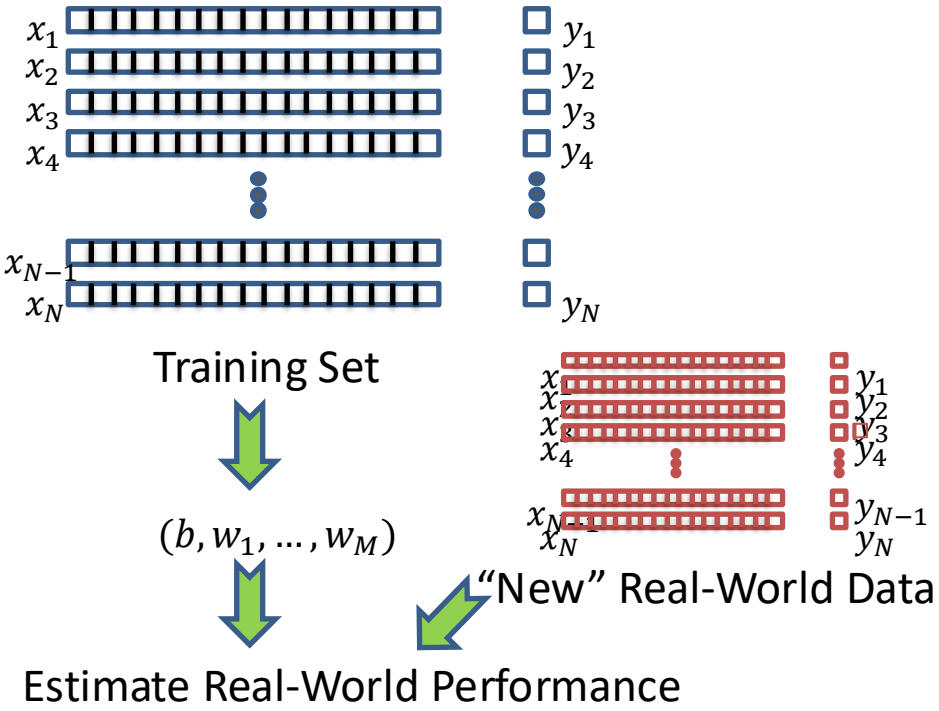

Training Set

$(b, w_1, \ldots, w_M)$

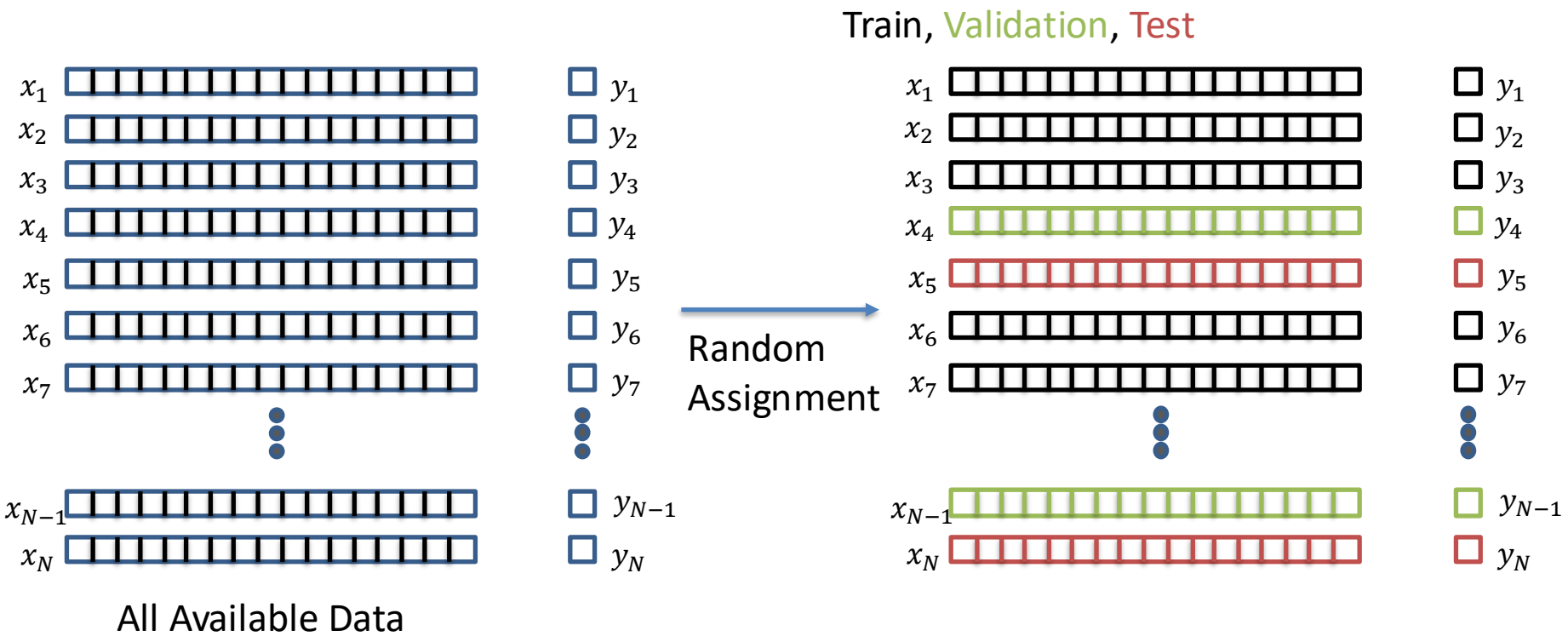
Performance in the Real World??

# Standard Validation Strategy

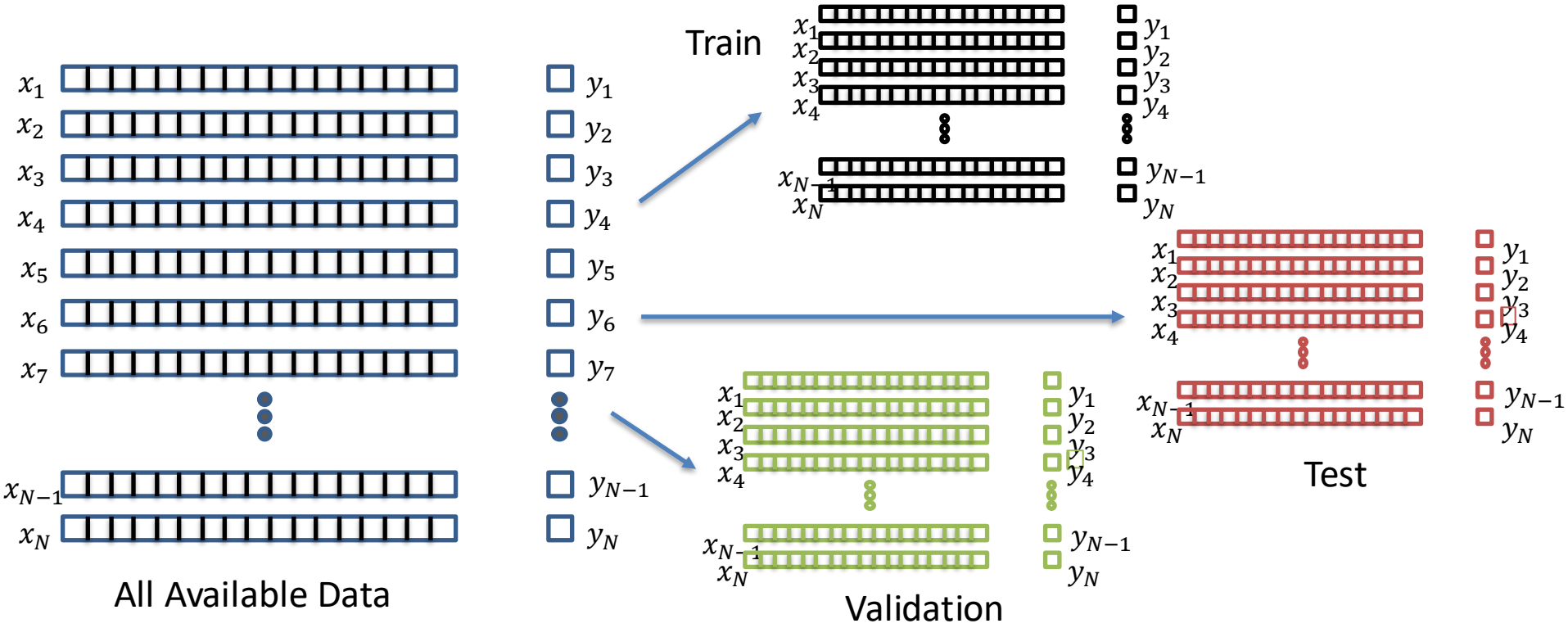
- In the end, we want to know how the network will perform *in the real world*
- Standard approach: actually, try it in the real world
- This is costly; instead, can we use existing data to estimate performance?



# Split Data into Separate Groups



# Split Data into Separate Groups



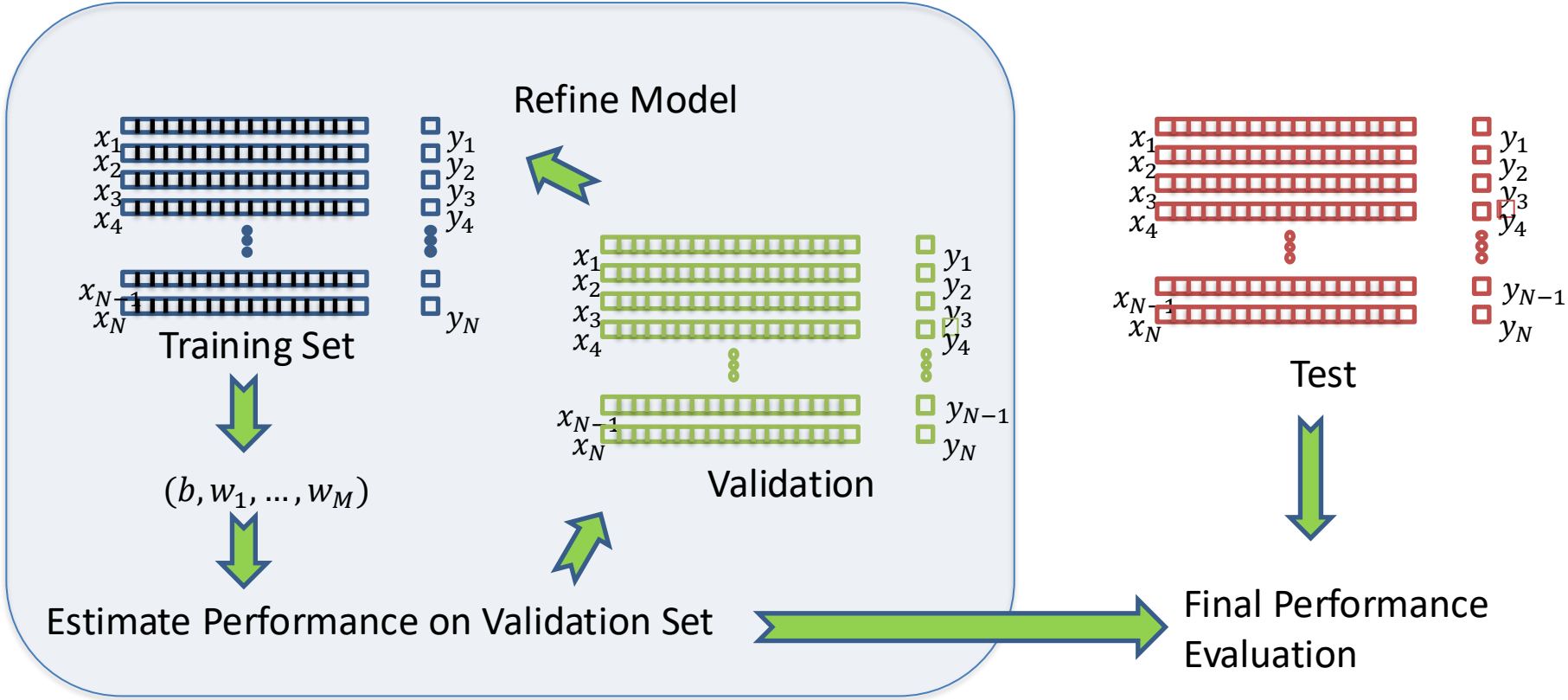
# Test set

- Standard practice to create prior to any analysis—**will not be used to learn or fit any parameters**
- After learning the network, can evaluate its performance on the test set
  - This data was not included in the training/fitting, so it is analogous to running a new synthetic experiment

- Ideally, the test set will be used *once*.
  - Reusing the test set leads to bias; performance estimates will be optimistic

# Validation Set

- Want to be able to compare which approach is best
  - Problematic if we only want to use a test set once
  - Can create a second held-out dataset
- The validation data is not used for learning parameters, but can be used repeatedly to estimate performance of a model
- We can pick the model with the best performance on the validation set, and run a final evaluation on the test data

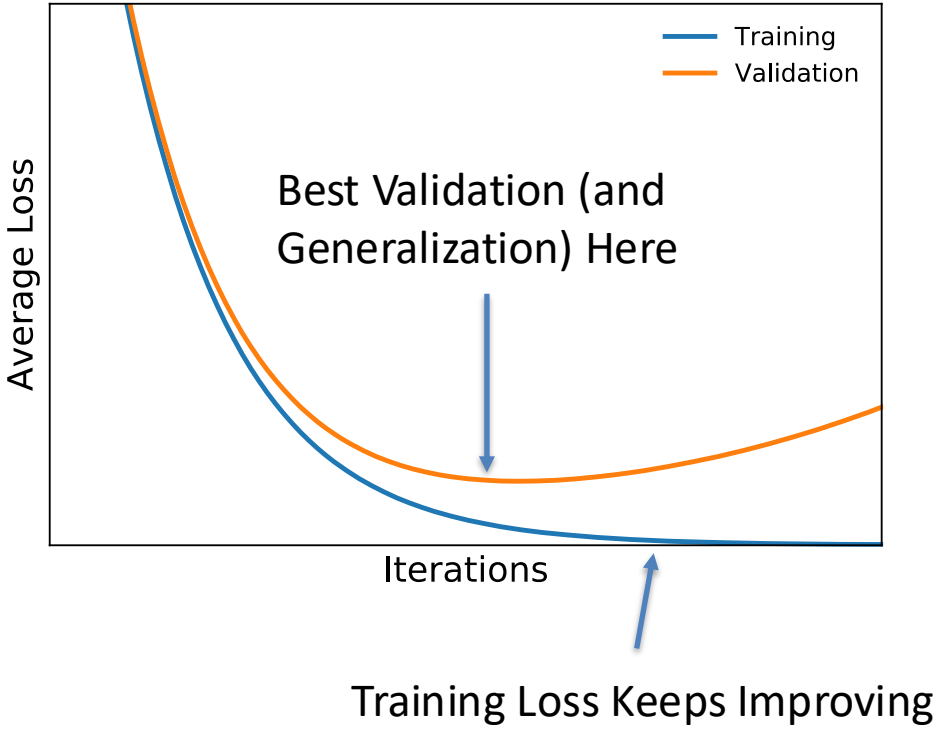

Refine Model
$x_1$ $x_2$ $x_3$ $x_4$ $x_{N-1}$ $x_N$
$y_1$ $y_2$ $y_3$ $y_4$ $y_N$
Training Set
$(b, w_1, \ldots, w_M)$
Estimate Performance on Validation Set
$x_1$ $x_2$ $x_3$ $x_4$ $x_{N-1}$ $x_N$
$y_1$ $y_2$ $y_3$ $y_4$ $y_{N-1}$ $y_N$
Validation
$x_1$ $x_2$ $x_3$ $x_4$ $x_{N-1}$ $x_N$
$y_1$ $y_2$ $y_3$ $y_4$ $y_{N-1}$ $y_N$
Test
Final Performance Evaluation

# Validation During Optimization

- We want to maximize the *generalization* of the network, not our previous optimization goal
- In practice, can we validate the model while running the optimization loop?

# Early Stopping

- During optimization, we can check the validation loss as we go.
- Instead of optimizing to convergence, we can optimize until the *validation* loss stops improving
  - Saves computational cost
  - Performs better on validation (and test) sets
- Widely used technique in the field



# Recall Learning Methodology

- Consider supervised learning, for simplicity, but similar setup will hold for the unsupervised learning of most NLP applications

- Learning performed with labeled data $\{(x_i, y_i)\}_{i=1,N}$ where $x_i \in R^d$ and $y_i \in R$

- Typically have a model for probability of $y_i$ given $x_i$: $p_\theta(Y = y_i | X = x_i)$ where $\theta \in R^M$ are the model parameters

- Simple example, where $y_i \in \{0,1\}$ and $\theta = (w, b)$:

$$p_\theta(Y = y_i | X = x_i) = [\sigma(w \cdot x_i + b)]^{y_i}[1 - \sigma(w \cdot x_i + b)]^{1-y_i}$$

# Cross-Entropy Loss

- Let $\mathrm{q}(\mathrm{X},\mathrm{Y}) = q(Y|X)q(X)$ represent the true (and unknown) distribution of the underlying system, from which training data sampled: $X$ covariates and $Y$ outcome

- For model $p_\theta(Y|X)$ the expected loss, which we seek to minimize, is

$$loss = -E_{q(Y|X)q(X)} \log p_\theta(Y|X)$$

- Approximate expectation using samples from training data, $(x_i, y_i) \sim q(Y|X)q(X)$

$$\ell(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(Y = y_i|X = x_i)$$

# Entropy & Cross-Entropy

- The entropy of distribution $p(X,Y)$ is

$$H(X,Y) = -\mathbb{E}_{p(X,Y)}\left[\log p(X,Y)\right]$$
$$= -\int dX dY\, p(X,Y) \log p(X,Y)$$

- Cross entropy between $q(X,Y)$ and $p(X,Y)$ is

Model

Sample from dataset

Model-independent distribution

$$CE(q\|p) = -\mathbb{E}_{q(X,Y)}[\log p(X,Y)] = -\mathbb{E}_{q(Y|X)q(X)}[\log p(Y|X)p(X)]$$
$$= -\int dX dY\, q(X,Y) \log p(X,Y)$$

- Using *samples* from $q(X,Y)$, cross entropy is the expected negative log-likelihood of our model, represented by $p(X,Y)$

# Entropy & Cross-Entropy

- Cross entropy between $q(X,Y)$ and $p(X,Y)$ is

$$CE(q\|p) = -\mathbb{E}_{q(X,Y)}[\log p(X,Y)] = -\mathbb{E}_{q(Y|X)q(X)}[\log p(Y|X)p(X)]$$

- Using *samples* from $q(X,Y)$, cross entropy is the expected negative log-likelihood of our model, represented by $p(X,Y)$

Sample from dataset $\quad (x_i, y_i) \sim q(Y|X)q(X)$

Calculate loss $\quad \ell(\theta_k) = -\frac{1}{N}\sum_{i=1}^{N} \log p_{\theta_k}(Y = y_i | X = x_i)$

Update model (parameters) $\quad \theta_{k+1} = \theta_k - \alpha_k \nabla \ell(\theta_k)$

Repeat until convergence

$|\ell(\theta_{k+1}) - \ell(\theta_k)| < \epsilon$

# Revisit Gradient Descent

(Doesn’t work well in practice with neural networks)

# Naïve Gradient Descent

- Basic gradient descent learning of the parameters proceeds as

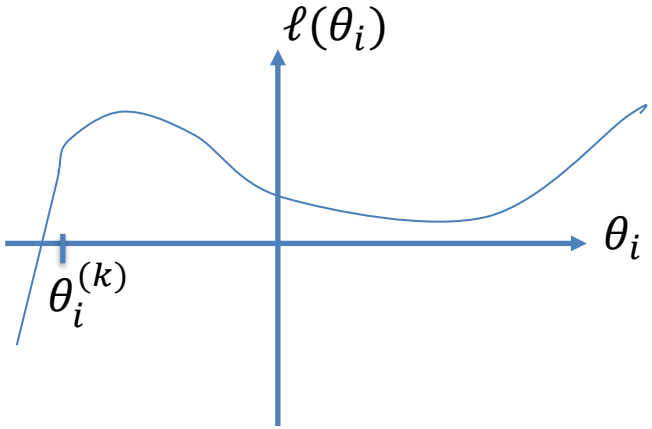
$$\theta_{k+1} = \theta_k - \alpha \nabla_\theta \ell(\theta)_{|\theta_k}$$

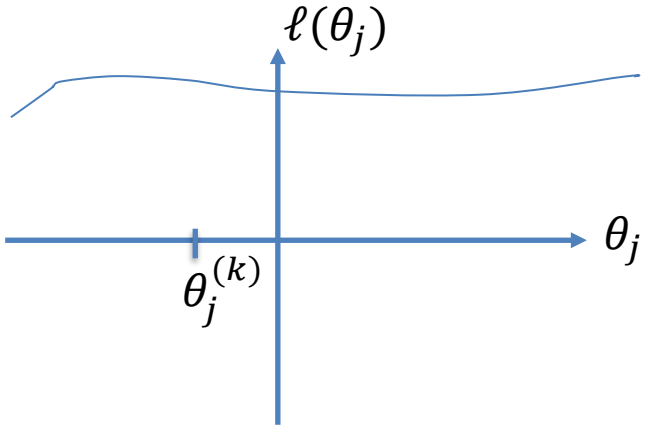
- We sequentially move in parameter space such that we reduce the loss

- Challenge: Neural networks have very large parameter vectors $\theta$, and the sensitivity of $\ell(\theta)$ to variation in parameters is often not the same for each component

- $\ell(\theta)$ more sensitive to some components of $\theta = (\theta_1, \dots, \theta_M)$ than others and which components of $\theta$ are important may change as a function of $\theta$

# Parameter-Dependent Variation in Derivative of Cost Function




- Difficult to have a single learning rate $\alpha$ with such variation in parameter-dependent derivatives (above are *components* $\theta_i$ and $\theta_j$ of $\theta$ at gradient-descent learning iteration $k$)

- Strong sensitivity of loss to some components of $\theta$, and weak for others, and the dependencies change as a function of $\theta$; hard to set a single learning rate for all components

# Pre-Conditioned Gradient Descent

- Modify gradient descent via

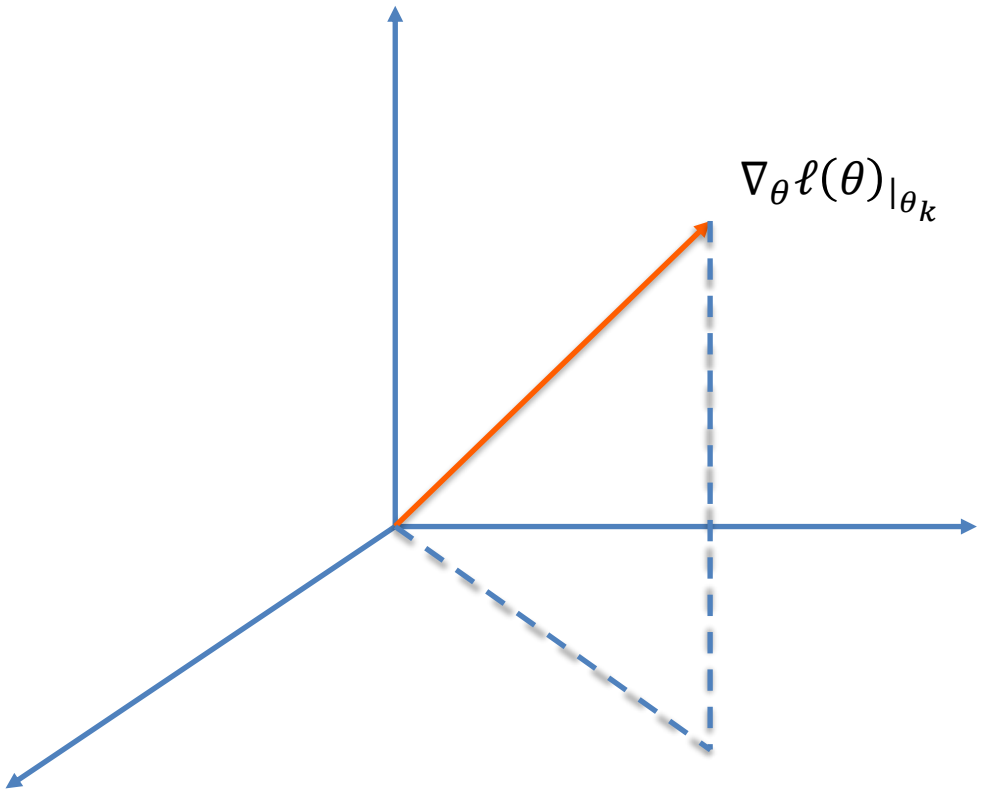
$$\theta_{k+1} = \theta_k - \alpha P \, \nabla_\theta \ell(\theta)_{|\theta_k}$$

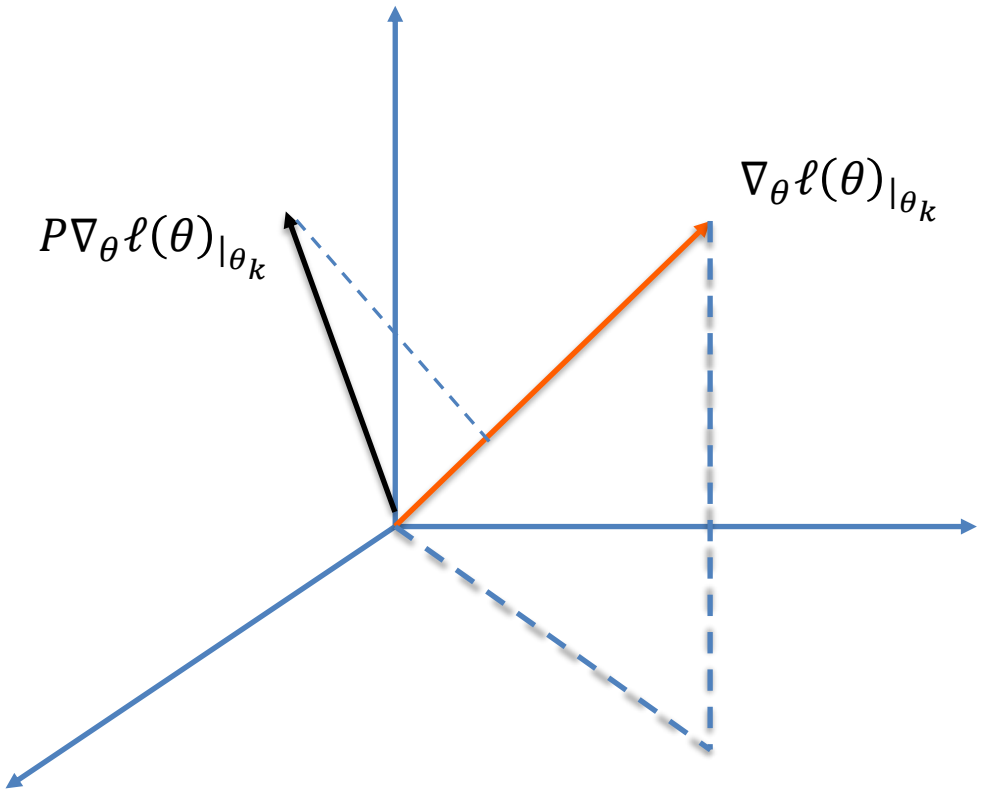
where $P$ is an $M \times M$ positive-definite matrix, recalling $\theta = (\theta_1, \dots, \theta_M)$

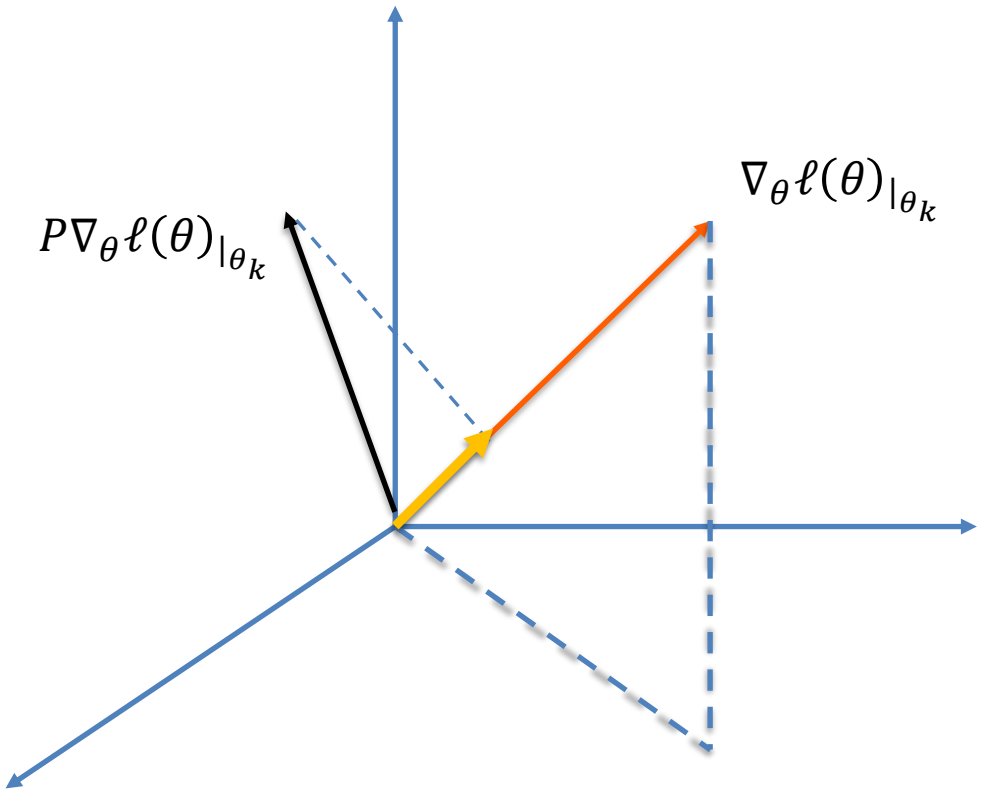
- Positive definite means

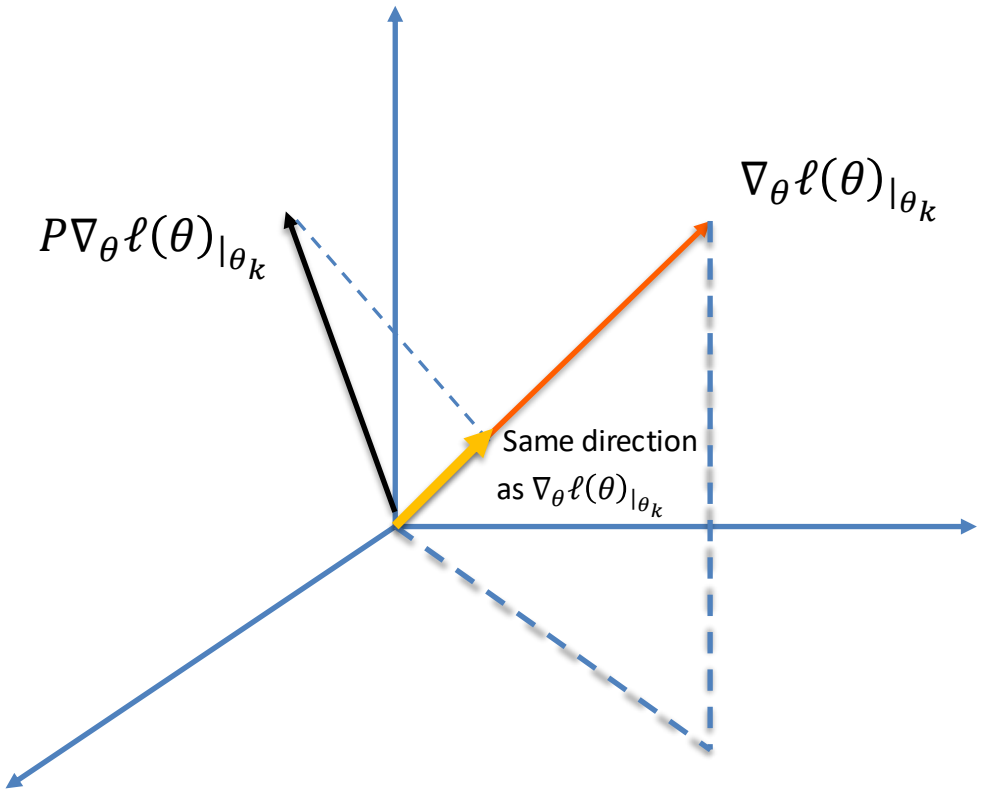
$$\nabla_\theta \ell(\theta)_{|\theta_k} \cdot P \nabla_\theta \ell(\theta)_{|\theta_k} > 0$$

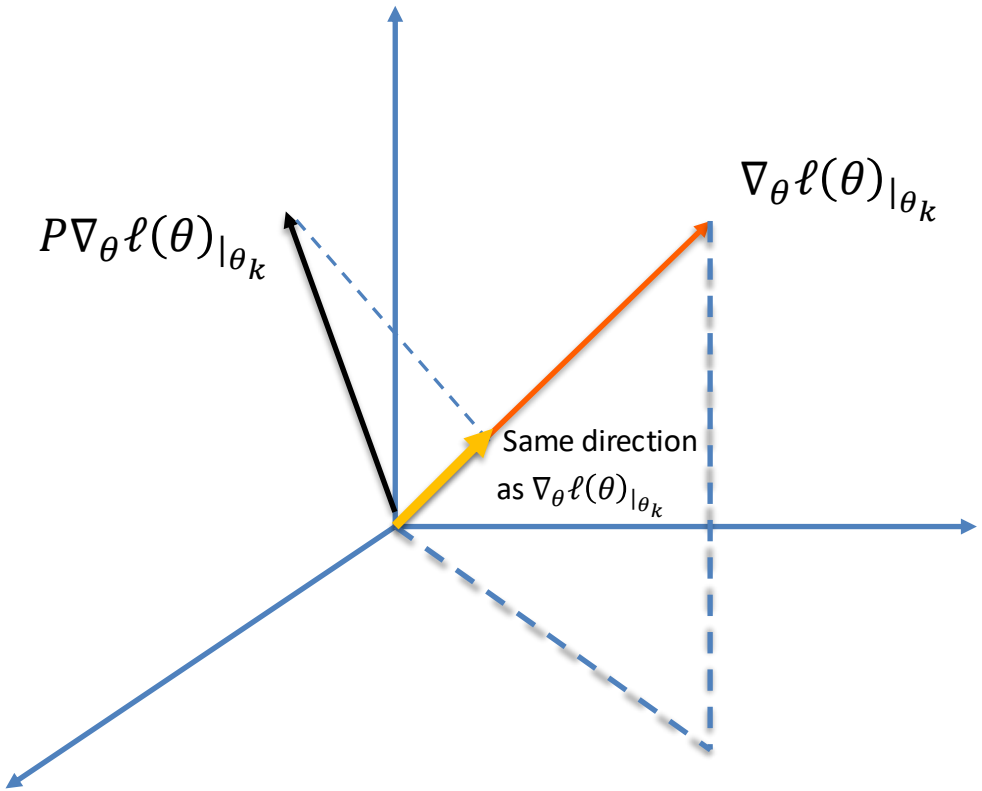
- This says that after preconditioning the gradient, parameter-update $\alpha P \, \nabla_\theta \ell(\theta)_{|\theta_k}$ is aligned with original gradient $\nabla_\theta \ell(\theta)_{|\theta_k}$


$\nabla_\theta \ell(\theta)|_{\theta_k}$


$P\nabla_\theta \ell(\theta)|_{\theta_k}$
$\nabla_\theta \ell(\theta)|_{\theta_k}$


$P\nabla_\theta \ell(\theta)|_{\theta_k}$
$\nabla_\theta \ell(\theta)|_{\theta_k}$


$P\nabla_\theta \ell(\theta)|_{\theta_k}$
$\nabla_\theta \ell(\theta)|_{\theta_k}$
Same direction
as $\nabla_\theta \ell(\theta)|_{\theta_k}$




$$\nabla_\theta \ell(\theta)|_{\theta_k} \cdot P\nabla_\theta \ell(\theta)|_{\theta_k} > 0$$

# RMSprop

- There are many different designs for the preconditioner $P$, and RMSprop is popular, and has motivated newer approaches

A mini-batch really

- Define the gradient vector at iteration $k$: $$g_k = -\frac{1}{N}\sum_{i=1}^{N} \nabla_\theta \log p_\theta(y_i|x_i)_{|\theta_k}$$

- Preconditioner continuously updated: $$P_{k+1} = diag\left(\left(\lambda + \sqrt{V_{k+1}}\right)^{-1}\right)$$

$$V_{k+1} = \gamma V_k + (1-\gamma) g_k \odot g_k\,, \qquad \gamma \in (0,1)$$

$V_{k+1}$ is an estimate of the variance of the gradient and $\odot$ represents Hadamard product

- Preconditioned gradient descent: $\theta_{k+1} = \theta_k - \alpha P_k g_k$

# RMSprop - Explanation

$$\theta_{K+1} = \theta_k - \alpha P_k g_k$$

$$g_k = \frac{-1}{N}\sum_{i=1}^{N} \nabla_\theta \log p_\theta(y_i|x_i)_{|\theta_k}$$

$$V_{k+1} = \gamma V_k + (1-\gamma) g_k \odot g_k$$

$$P_{k+1} = diag\left(\left(\lambda + \sqrt{V_{k+1}}\right)^{-1}\right)$$

- Analogous to parameter-dependent learning rate
- Quick rate when parameter derivative small, slow rate when derivative large
- Critical for complicated "landscape" of cost function $\ell(\theta)$ as a function of $\theta$

# RMSprop – Motivation for Extension

$$\theta_{K+1} = \theta_k - \alpha P_k g_k$$

$$g_k = \frac{-1}{N}\sum_{i=1}^{N} \nabla_\theta \log p_\theta(y_i|x_i)_{|\theta_k}$$

$$V_{k+1} = \gamma V_k + (1-\gamma) g_k \odot g_k$$

$$P_{k+1} = diag\left(\left(\lambda + \sqrt{V_{k+1}}\right)^{-1}\right)$$

- In RMSprop, we have a moving average of the variance of gradients, but compute gradient separately at each step
- Extension 1: Introduce a running average of gradients
- Extension 2: Introduce bias correction

# Track the Mean & Variance of Gradient

$$g_k = \nabla_\theta \ell(\theta)_{|\theta_k}$$

- It is natural to track both the mean and simple variance of the gradient

$$m_k = \beta_1 m_{k-1} + (1-\beta_1) g_k$$

$$v_k = \beta_2 v_{k-1} + (1-\beta_2) g_k \odot g_k$$

- The update rule for the preconditioned gradient descent is

$$\theta_k = \theta_{k-1} - \alpha m_k / \sqrt{\lambda + v_k}$$

Conceptually:
normalizing the gradient!

where the division is performed element-wise and $\beta_1 \in (0,1)$, $\beta_2 \in (0,1)$

# Written Differently

- Using the update equations below for the mean and variance

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k$$

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k \odot g_k$$

we can show that

$$m_k = (1 - \beta_1) \sum_{i=0}^{k-1} \beta_1^i g_{k-i}$$

$$v_k = (1 - \beta_2) \sum_{i=0}^{k-1} \beta_2^i g_{k-i} \odot g_{k-i}$$

# Performing Expectation

- The training data are random draws, and the mean batches are randomly selected

$$\mathbb{E}\left[g_{k-i}\right] \approx \mathbb{E}[g_k] + \epsilon_i$$

- Look at expectations of mean and variances:

$$\mathbb{E}[m_k] = (1-\beta_1)\sum_{i=0}^{k-1}\beta_1^i\mathbb{E}[g_{k-i}] \approx \mathbb{E}[g_k](1-\beta_1)\sum_{i=0}^{k-1}\beta_1^i + \xi_m$$

- The residual/error $\xi_m$ should be small for $\beta_1$ near one (local average)

$$\mathbb{E}[g_{k-i} \odot g_{k-i}] \approx \mathbb{E}[g_k \odot g_k] + \delta_i$$

- Similarly

$$\mathbb{E}[v_k] \approx \mathbb{E}[g_k \odot g_k](1-\beta_2)\sum_{i=0}^{k-1}\beta_2^i + \xi_v$$

# Self Exercise

$$\mathbb{E}[m_k] \approx \mathbb{E}[g_k](1-\beta_1)\sum_{i=0}^{k-1}\beta_1^i + \xi_m$$

$$\mathbb{E}[v_k] \approx \mathbb{E}[g_k \odot g_k](1-\beta_2)\sum_{i=0}^{k-1}\beta_2^i + \xi_v$$

We can show that

$$(1-\beta)\sum_{i=0}^{k-1}\beta^i = (1-\beta^k)$$

# Continuing With the Analysis

$$\mathbb{E}[m_k] \approx \mathbb{E}[g_k](1-\beta_1)\sum_{i=0}^{k-1}\beta_1^i + \xi_m$$

$$\mathbb{E}[v_k] \approx \mathbb{E}[g_k \odot g_k](1-\beta_2)\sum_{i=0}^{k-1}\beta_2^i + \xi_v$$

$$\mathbb{E}[m_k] \approx \mathbb{E}[g_k]\left(1-\beta_1^k\right) + \xi_m$$

$$\mathbb{E}[v_k] \approx \mathbb{E}[g_k \odot g_k](1-\beta_2^k) + \xi_v$$

# Continuing With the Analysis

$$\mathbb{E}[m_k] \approx \mathbb{E}[g_k](1-\beta_1)\sum_{i=0}^{k-1}\beta_1^i + \xi_m$$

$$\mathbb{E}[v_k] \approx \mathbb{E}[g_k \odot g_k](1-\beta_2)\sum_{i=0}^{k-1}\beta_2^i + \xi_v$$

Bias

$$\mathbb{E}[m_k] \approx \mathbb{E}[g_k]\left(1-\beta_1^k\right) + \xi_m$$

$$\mathbb{E}[v_k] \approx \mathbb{E}[g_k \odot g_k](1-\beta_2^k) + \xi_v$$

# Adam

$$m_k = \beta_1 m_{k-1} + (1-\beta_1) g_k$$

$$v_k = \beta_2 v_{k-1} + (1-\beta_2) g_k \odot g_k$$

Moving Average

$$\hat{m}_k = m_k / (1-\beta_1^k)$$

$$\hat{v}_k = v_k / (1-\beta_2^k)$$

Bias Correction

$$\theta_k = \theta_{k-1} - \alpha \hat{m}_k / \sqrt{\lambda + \hat{v}_k}$$

Parameter Update

# Adam

D.A. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization,: 2014

**Adam**: A method for stochastic optimization
DP **Kingma**, J Ba - arXiv preprint arXiv:1412.6980, 2014 - arxiv.org
… **Adam** works well in practice and compares favorably to other stochastic optimization methods. Finally, we discuss AdaMax, a variant of **Adam** … Overall, we show that **Adam** is a versatile …
Save Cite Cited by 234363 Related articles All 15 versions

$$m_k = \beta_1 m_{k-1} + (1-\beta_1) g_k$$

$$v_k = \beta_2 v_{k-1} + (1-\beta_2) g_k \odot g_k$$

Moving Average

$$\hat{m}_k = m_k / (1-\beta_1^k)$$

$$\hat{v}_k = v_k / (1-\beta_2^k)$$

Bias Correction

$$\theta_k = \theta_{k-1} - \alpha \hat{m}_k / \sqrt{\lambda + \hat{v}_k}$$

Parameter Update

# Start Moving Toward NLP